Show Me Your ID! - code guide and Q&A prep

Written for: the person who has to answer questions about how the game plays and how the code works (a panel, a class defense, a teammate). Checked against the code on `main` on November 3, 2026. Read parts 1 and 2 first. Then use parts 9 and 10 as flash cards.

`PITCH-GUIDE.md` is the shorter study guide (brought up to date the same day). This one goes further and every number here was re-checked in the code.

---

1. "Why is there no camera in the Hierarchy?" (read this one twice)

The short answer to say out loud

> "The camera is not saved in the scene. The game creates it with code the moment it starts, once, and keeps it alive across all three scenes. Press Play and you will see it under DontDestroyOnLoad. We did it this way so there is only ever one camera, one audio listener and one game controller, and no scene can be missing them or have two."

What you actually see in Unity

When	What the Hierarchy shows	Why
Edit mode, a scene that was never set up	One object only: `MenuScene`, `DeskScene` or `PatrolScene`	Each scene file holds just one small script object that starts the game
Edit mode, after **Hollowbrook > Set Up Project**	The above, plus `SceneRoot_MenuScene` (or `_DeskScene`, `_PatrolScene`) with the canvases, screens, buttons and labels inside	The tool saves the user interface into the scene so it can be edited
Edit mode, either way	**No camera, no GameRoot, no EventSystem, no AudioDirector**	They are made by code at runtime, not saved in any scene
Play mode	A fourth section called **DontDestroyOnLoad** appears, holding `GameRoot`, `AudioDirector`, `EventSystem` and `MainCamera`	The code marks them to survive scene changes

In Edit mode the Game view says **"Display 1: No cameras rendering"**. That is expected, not a bug.

Why we built it this way

1. **One camera for the whole game.** The desk, the hallway and the main menu are three different scenes. If each scene had its own camera, Unity would swap cameras on every scene change (a flicker or a blank frame) and we could end up with two audio listeners (Unity warns about that). One camera that survives every scene avoids both.
2. **The camera cannot be deleted or set up wrong.** Its settings are in code, so nobody can change them by accident in the Inspector: orthographic (flat 2D, no perspective), size 5 (so the screen is 10 world units tall), solid black background, placed at (0, 0, -10) with an audio listener. See `Scripts/Scenes/GameCore.cs`, method `EnsureCore`, line 71.
3. **Most of the game does not need a camera at all.** The desk, menus, dialogue, HUD, results and ending are all uGUI canvases set to **Screen Space - Overlay**. Overlay canvases

are drawn straight on the screen with no camera. The camera only matters in the patrol (hallway and rooms), where sprites live in the world.

4. **Same idea for the controller.** `GameRoot` holds the whole run (the night, sanity, the save, links to every screen). It has to survive scene changes, so it is created once by code and never lives in a scene file. Code never searches for objects by name. It asks `GameRoot`.

What the camera does while playing

- **Desk scene:** nothing to see. The camera shows black behind the overlay canvases.
- **Patrol scene:** `ViewManager` moves the camera sideways to follow the guard down the hallway (`World/ViewManager.cs`, around line 383). While you peek into a room the camera is frozen at the center (line 295) and the black cover with the lamp hole is drawn over the room.
- The flashlight and the peek lamp read the mouse position by projecting it onto the flat z = 0 plane of the world (`FlashlightRig.cs` line 200, `PeekLight.cs` line 273).

Prove it live in the Q&A (30 seconds)

1. Open `MenuScene.unity`. Point at the Hierarchy: only the scene object. Say "nothing here is the camera".
2. Press Play. The **DontDestroyOnLoad** section appears. Point at `MainCamera`, click it, and show the Inspector: Orthographic, Size 5, black background.
3. Click NEW GAME. Say "the scene changes, the camera does not: it is the same object".

The honest downside (say it before they ask)

"The camera and the controller cannot be edited visually in the scene. An artist has to ask a programmer to change them. We accepted that for stability, and we moved all the screens (the part artists touch) into the scene files with the Set Up Project tool."

Other "where is it?" questions with the same answer

"Where is the...?"	Answer
Player / guard	Made by code when the patrol starts (`ViewManager` creates `PlayerController`). Only exists in PatrolScene while playing
Hallway and rooms	Built from code and data (`RoomBuilder`, `FloorPlan`) under an object called `World`, which starts switched off
Doors, lockers, clues	Data objects (`Interactables.cs`) built each time the patrol loads
Visitors, papers, stamps	Made while playing. The desk shows whoever the roster says is at the window
Anomalies	Spawned during the patrol (`RoomSpawner`), they do not exist in any scene
Buttons and labels	Yes, these ARE in the scene files after Set Up Project (under `SceneRoot_...`)
Colliders / physics	There are none. The game has no Rigidbody or Collider anywhere (see part 6, patrol)

---

2. The game in one page (how it plays)

Job: you are the night custodian of Hollowbrook College. Five nights in a row you check who is allowed in. Some visitors are not students.

One night:

1. The night title and a short dialogue play, then the booth opens.
2. Visitors arrive one by one on an arrival timer (25 to 30 seconds on night 1, 0.8 times that on each later night). When one reaches the window you check the **ID card**, the **paper slip**, the **face** against the ID photo, and the **student file** in the cabinet.
3. You may ask **2 of 4 questions** (course, year, regular or transferee, first classroom). Year unlocks after you flip the ID. Status unlocks after you open the file.
4. You drag **ACCEPT** or **DECLINE** onto the slip, then drag the ID back through the window. The decision is permanent.
5. Between visitors you can **patrol**: walk the dark hallway with a flashlight, peek into rooms with a lamp, find clues, and push away roaming anomalies. Each night needs one patrol cycle (Room 101, Room 202 and one comfort room) before END SHIFT.
6. END SHIFT. The next night starts straight away (there is no main menu in between). You can pause and quit at any time, and Continue resumes from the save.

Night 5: the last visitor is the district administrator (the final anomaly). You question him with `T`, then stamp. DECLINE starts the **hunt**: he gets into the building and you survive until dawn using only the lamp. ACCEPT loses at once.

After the story: Night 6 (the Challenge, a harder hunt) and **Lockdown** (13 frightened students, 35 seconds each, no patrol) open.

Win / Lose / Secret: three endings. The Secret needs all ten clues found in order.

Why it works as a game: every visitor is a small puzzle with checkable evidence, two questions out of four force a choice, and breaches (letting an anomaly in) and false rejections (turning a real student away) both hurt, so "accept everyone" and "decline everyone" both fail.

---

3. The big picture of the code

- **Engine and tools:** Unity 6.3 (6000.3.0f1), C#, Built-in Render Pipeline, uGUI for the interface, the the Input System package (1.20.1) with Active Input Handling set to Both: the code reads keys and the mouse with the classic `Input` class, and the UI clicks go through the Input System UI Input Module.
- **Size:** 130 C# scripts, about 22,600 lines, all under `Assets/_Project`.
- **3 scenes:** `MenuScene`, `DeskScene` (the desk), `PatrolScene` (the patrol). Each file holds one object with the same name, the script that starts that scene.

The one idea: rules are separate from screens

Layer	Folder	Knows about Unity?	In plain words
Rules (the brain)	`Scripts/GameLogic`	**No**, plain C#	Answers questions: "is this ID expired?", "which ending is it?"
Orchestrator	`Scripts/Core`	Yes	`GameRoot` ties the rules to the screens, owns the save and the audio
Screens (the body)	`Checkpoint`, `UI`, `World`, `Horror`	Yes	Buttons, the desk, rooms, the flashlight, scares
Scene setup	`Scripts/Scenes`	Yes	Creates the core and finds or builds each scene's screens

Layer	Folder	Knows about Unity?	In plain words
Placeholders	`Scripts/Generated`	Yes	Code-painted pictures used only when no real art file exists
Audio	`Scripts/Audio`	Yes	Plays sounds from real audio files
Editor only	`Editor/ProjectSetup.cs`	Yes	The Hollowbrook menu in Unity. Never part of the built game

Why this matters: the rules never say "draw a box". They only answer questions. Screens ask the rules and draw the answer. So we can test the whole game without opening Unity, and change art without touching rules.

Patterns you can name

- **`GameRoot` (a singleton-style central object):** one object, found through `GameRoot.Instance`, that holds the run and links to every screen.
- **Partial classes:** `GameRoot` and `CheckpointView` are one class each, split across many files by topic (`GameRoot.Hunt.cs`, `GameRoot.Saving.cs`, `CheckpointView.Papers.cs`…). The compiler sees one class. We did it so no file is huge and each file has one topic.
- **EventBus:** parts publish a message with a text id (`Bus.Publish(...)`) and anything interested subscribes. The audio, the HUD and the caption bar listen without anyone calling them directly.
- **State machines:** `AppViewState` (menu, checkpoint, patrol, pause…), `HuntPhase` (stalking, in a room, behind you, loose, retreat, won, lost), and the tutorial steps.
- **Data in one place:** `GameRules.cs` and `GameRules.Tables.cs` hold every rule number, the five nights and the students as table rows. `GameConfig.asset` is the live copy that can be edited in the Inspector.

---

4. What happens when you press Play (follow the chain)

1. Unity opens `MenuScene` (the setup tool makes Play always start there). The scene holds a `MenuScene` component. Its `Awake` calls `GameCore.EnsureCore()`.
2. **`EnsureCore`** (first time only) creates the shared things in code and marks them to survive scene changes: `AudioDirector`, `GameRoot`, `EventSystem` (needed for every uGUI click) and `MainCamera`. It also builds the `EventBus`, loads `GameConfig`, `SettingsSystem` and `SaveSystem`.
3. `MenuScene.Build` finds or builds the menu screens under `SceneRoot_MenuScene`, then `root.BootToMainMenu()`.
4. You click NEW GAME. `GameRoot.StartNewGame` makes a fresh `GameSession` (with a random seed) and **queues** an action ("start night 1"), then loads `DeskScene` through `SceneRouter`.
5. In the new scene, `DeskScene.Awake` calls `EnsureCore` again (it finds the existing root and does nothing), builds the desk screens, and runs the queued action. That calls `BeginShift`, which makes the night's roster (`RosterFactory`), the rules for the night (`ShiftPlan`) and plays the intro.
6. Visitors arrive (`CheckpointQueue` timer) and the desk (`CheckpointView`) shows them.

7. You press PATROL. `GameRoot.Patrol` queues "start patrol" and loads `PatrolScene`, whose script builds the `World` object with `ViewManager`, which creates the rooms, the guard, the flashlight and the spawner. The camera (already alive) starts following the guard.
8. END SHIFT saves the night and starts the next one. After night 5 (or a loss) the ending screen shows.

Safety nets worth mentioning:

- If a scene loads without its scene script, the queued action is never consumed. After 3.5 seconds `GameRoot` notices ("scene handshake failed"), says so, and goes back to the menu instead of freezing (`GameRoot.cs`, `CheckSceneHandshake`).
- If anything throws while a scene is being built, `GameCore.SafeBuild` logs it, shows a message and returns to the menu.
- `EnsureSceneStarted` loads the main menu if the game starts in a scene with no `GameRoot`.

---

5. File-by-file tour

Open the file names in this order when someone asks "show me". Sizes are lines.

`Scripts/Core` (the orchestrator, 17 files)

File	What it is
`GameRoot.cs` (378)	The central object. Holds services, the run, links to every screen, the shift clock, the scene handshake
`GameRoot.Shifts.cs` (317)	Starting and continuing a run, beginning a night, the queue
`GameRoot.Patrol.cs`	Leaving the desk for the patrol, peeking into rooms, coming back
`GameRoot.Hunt.cs`	Night 5 and 6: after DECLINE the window closes, the hunt starts when Patrol is clicked
`GameRoot.Endings.cs`	How a night ends: END SHIFT, a failed night, a blackout, the endings
`GameRoot.Saving.cs`	Writing the save, erasing data
`GameRoot.Lockdown.cs`, `.Tutorial.cs`, `.Unlocks.cs`	The Lockdown mode, the Night 1 tutorial, which modes are open
`EventBus.cs`, `EventIds.cs`	The message system and the list of message names
`GameConfigSO.cs`	The tuning asset (queue, battery, hunt, anomaly speeds)
`SaveSystem.cs`	Reads and writes the save files on disk
`SettingsSystem.cs`	Volumes and screen mode, stored in PlayerPrefs
`InputGate.cs`	Makes sure only one view reacts to a key per frame (Esc and M are shared)
`AspectMatchGuard.cs`	Keeps every canvas correct on any screen shape (reference 1280 x 720)
`IScreenView.cs`	The small interface every screen implements

`Scripts/Scenes` (how scenes start, 6 files)

File	What it is
`GameCore.cs`	**Creates the camera, GameRoot, EventSystem and audio.** Also the scene helpers, the scene router and the safety nets
`MenuScene.cs`, `DeskScene.cs`, `PatrolScene.cs`	The one component on each scene. They connect the screens to the game when the scene loads
`SceneBuild.cs`	"Find or make": reuse an object already in the scene, make it only if missing

File	What it is
SceneEdits.cs	Remembers what the code made, so edits you make in the scene file are put back on top

Scripts/GameLogic (the rules, 46 files, no Unity)

File	What it is
GameRules.cs, GameRules.Tables.cs	Every rule number, the five nights and the 13 students as rows
GameSession.cs	Everything that has happened this run: night, sanity, breaches, the administrator state. This is what is saved
ShiftPlan.cs, ShiftFailureRules.cs	The nights as data, and what a failed night costs the next one
RosterFactory.cs (525)	Makes the visitors for a night and gives anomalies their defects. This is where students lie
IdRules.cs, InspectionRules.cs, Questions.cs	What is right or wrong on the papers, and the four questions
CheckpointQueue.cs (498)	The arrival line: timer, who is waiting, who is at the window
EndingRules.cs	Pure function from the session to an ending
AdminHunt.cs (469), AdminTalk.cs (366)	The hunt state machine and the administrator's questions and lies
AnomalyBehaviorRules.cs, AnomalyEncounter.cs, TrackingPolicy.cs	How hallway anomalies move, threaten and get repelled
PatrolCycle.cs, PatrolPool.cs, PeekSpots.cs	What a patrol requires, which anomalies wander, what stands in peek rooms
BeamGeometry.cs, BeamProfile.cs, BatteryModel.cs, FlashlightInput.cs, HallwayDarkness.cs	The flashlight maths, battery and hallway darkness
SanityStacks.cs, NightLog.cs, ClueRegistry.cs	Sanity drain, the Security Log, the ten clues
Lockdown.cs, TutorialSteps.cs, UnlockCodeReader.cs	Lockdown rules, tutorial steps, the main-menu unlock code
SaveFormat.cs (308)	Turns the session into text and back
FloorPlan.cs, MapSheetLayout.cs	The school map data and its layout maths
ContentData.cs, CreditsData.cs, FaceAssets.cs	Greetings and text, the credits list, which face art goes where
DrawerMotion.cs, StampPlacement.cs, SpeechBubble.cs, CaptionPolicy.cs, DisplayPolicy.cs, PresentationLog.cs, RecordSearch.cs, StudentData.cs, GameRng.cs, Enums.cs	Small helpers: drawer and stamp maths, bubble state, when captions may show, a seeded random number generator, the enums

Scripts/Checkpoint (the desk, 16 files)

CheckpointView is one class split by area: .Window (the visitor and the ID hand-over), .Papers (record sheet and ID card), .FileDrawer (the cabinet, 647 lines), .Interview (the four questions), .Decision (tools and stamps), .Speech (bubble and buttons), .Overlays (rules note and the log), .TopBar, .Sticky (the start-of-night note), .AdminTalk, .Lockdown, .KitSkin (the art kit placed over the layout). Also CheckpointLayout.cs (every desk position as numbers in a 1280 x 720 frame), DragPaper.cs (papers and stamps are draggable objects) and DeskLayoutAudit.cs (checks the built desk matches the grid).

Scripts/World (the patrol, 13 files)

File	What it is
ViewManager.cs	Rooms, doors, fades, anomaly spawning, the camera follow, the flashlight and the peek
PlayerController.cs	Walk, run, interact for the guard (walk cycle is four sprites)
RoomBuilder.cs, RoomInstance.cs, RoomSpawner.cs, Interactables.cs	Builds every room, door and clue, and spawns anomalies
FlashlightRig.cs, BeamCover.cs, PeekLight.cs, LightController.cs, WorldLayers.cs	The beam, the dark cover with a hole, the peek lamp, room lighting, draw order
PeekAnomaly.cs, AdminHuntDirector.cs	The standing figures in peek rooms, and the hunt shown in the world

Scripts/Horror, UI, Audio, Generated, Editor

Folder	Files
Horror	SanityController (screen effects by sanity stage), ScareSystem (jumpscares with accessibility guards), HallucinationSystem (false sightings and small wrong noises), WanderingAnomaly (631 lines, the roaming anomalies)
UI	UIFactory (builds every button and label so the game shares one look), MainMenuView, HUDView, DialogueView, MapView, EndingView, ShiftResultsView, PauseView, SettingsView, CreditsView, ControlsView, CaptionBar, ConfirmationPopup, LoadingView, TutorialGuide and a few small helpers
Audio	AudioDirector (plays sounds and music) and AudioLibrary (finds a sound by name in Resources/Audio)
Generated	ProceduralArt.* and FaceArt: placeholder pictures painted by code. A real file with the same name in Resources/Art always wins
Editor	ProjectSetup.cs: the **Hollowbrook** menu (Set Up Project, Check Project, Rebuild Scenes From Scratch)

Outside the game folder

Path	What it is
Assets/_Project/Resources/GameConfig.asset	The live tuning numbers, editable in the Inspector
Assets/_Project/Resources/Art, Fonts	Real art and fonts, loaded by name. There is no Resources/Audio folder yet
Tools/LogicTests, Tools/FlowCheck, Tools/Verify, Tools/run_all.sh	The automatic checks (part 7)
Tools/prep_*.py	Python scripts that prepare art for the game
docs/	This guide, the GDD, GUIDE.md (where do I change X), GAMEPLAY-OVERVIEW.md, ART-NAMING.md, ASSET-MANIFEST.md

---

6. How the main systems work

The inspection (the heart of the game)

Every visitor is a VisitorData: an ID card, a record in the drawer, a face and four answers. A real student matches everywhere. An anomaly has at least one defect, and every defect has a tell the tests prove can be found. There are **13 defect types** (DefectType in Enums.cs): photo mismatch, subtle photo mismatch, expired ID, bad ID format, year, course or status mismatch, name mismatch, not in the registry, duplicate ID, classroom lie, spoken lie and slip mismatch. Some show on the papers, some only in what they say.

Who is an anomaly

`RosterFactory` decides which visitors are anomalies each night (2, 3, 4, 5 and 5 on nights 1 to 5) and gives them defects. `IdRules` finds those defects from the papers. The stamp you give is compared with the truth the factory made. About half of the anomalies also speak awkwardly. Some visitors arrive scared (1, 2, 2, 3, 3 per night): a scared real student and a scared anomaly look and sound the same, so fear proves nothing.

The Security Log (nights 1 to 4)

Its boxes fill themselves in. Under it a slip lists everyone you let in. Before END SHIFT you may **report one** person. A right report cancels that one breach. A wrong report costs 5 sanity at the start of the next night. It is a second chance, not a chore.

Sanity

0 to 100, in four stages: Stable 70 and above, Uneasy 40, Shaken 15, Breaking below. It drains when a visitor stands at the window (0.25 per second), in the dark hallway or a peek (0.3 per second), when an anomaly is near (1.5 per second) and when one catches you (15). A correct decision gives back 8. Nights 5 and 6 drain slowly everywhere (twice as fast on night 6). Mistakes themselves do not cost sanity. At 0 on nights 1 to 4 you black out and that night is replayed. On night 5 it is the Lose ending. All numbers are in `GameRules.cs`.

Patrol, the flashlight and why there is no physics

- The hallway is one wide painted image in the dark. The guard carries only a weak light. The flashlight (F sweep, Q focus, L off) points at the mouse.
- `BeamGeometry` (plain maths) decides what the beam covers. `BeamCover` draws the darkness with that cone cleared out. The same shape decides what the beam slows or repels, so **what you see is what counts**.
- **No physics on purpose.** The guard moves with `Transform` positions in `PlayerController`. Doors and items work by distance along the hallway (`Interactable.DistanceTo` compares x positions), and the mouse selects nothing on patrol. This is simple, predictable and testable without Unity's physics.
- A battery drains while a beam or the peek lamp is lit (faster in focus mode) and is full at the start of each patrol.

Hallway anomalies and peek figures

Four behaviors (`AnomalyBehaviorKind`): Blink, Slide, Sprint and CoverHug. In the game: a crouched one that slides, a four-legged crawler that runs along the floor or ceiling (always fast), and one that hangs from the top of the hallway. The wide sweep only shows them and slows them to half speed. The **focus** beam, held on them, repels them. Peek rooms can hold a standing figure that never moves: hold the lamp on its body for about 3 seconds and it vanishes. If you step out of a peek and leave it standing, that door becomes a jumpscare the next time you open it.

Peeking

Rooms are never entered. From the doorway you move a lamp. The room is covered by a black sheet with one soft hole that follows the mouse (`PeekLight.cs`), so whatever is under the hole is seen. There is no light sprite.

From the door to the peek, step by step (all in `PatrolScene`):

1. **No clicking.** The mouse never selects a door on patrol, it only aims the flashlight. Walk the guard up to a door: `PlayerController` checks every door and item in the hallway each frame, picks the nearest one within `interactRange` (1.35 units, measured only along the hallway, left and right), and shows its prompt, for example "Peek into Room 101 [E]".
2. **Press E.** `PlayerController` calls `Interact()` on that door (`Doorway` in `World/Interactables.cs`). The security door goes back to the desk (or is locked during the hunt). Any other door calls `GameRoot.EnterRoomPeek(room)`.
3. **`GameRoot.EnterRoomPeek`** (`Core/GameRoot.Patrol.cs`) first checks for a waiting jumpscare (a figure the player left standing in that room last time). If there is one, the scare plays, costs sanity, uses the figure up, and the guard stays in the hallway. Otherwise it remembers which room is being peeked, ticks the room off the patrol cycle if it is empty, sets the state to `RoomPeek`, and asks `ViewManager.EnterPeek(room)`.
4. **`ViewManager.EnterPeek`** plays the door sound and fades to black. In the black it swaps the hallway for the room, hides the guard (he stays in the hallway behind the peek), turns the room's own light off, freezes the camera on the room, and turns on the peek lamp: the black cover with the hole. If the room holds a standing figure tonight, it is placed now. Then it fades back in. The player can act straight away.
5. **In the peek:** the mouse moves the hole, `F` turns the lamp on or off, `Q` switches wide or tight, `E` selects only what the lit lamp is on (a clue or an item), and `S` or the bottom edge steps back. Holding the lamp on a figure for about 3 seconds clears it, and the room then counts for the patrol.
6. **Stepping back** (`ViewManager.ExitPeek`): fade to black, a figure still standing makes that door a jumpscare for next time, the lamp and the figure are removed, the hallway comes back with the guard where he stood, and the state returns to `Exploration`. During the figure's vanishing sequence the player cannot step back until it ends.
7. **Night 5:** the same doors, but the administrator may be in the room. Keep the lamp on his head, or he catches you.

The administrator and the hunt (nights 5 and 6)

After the seven or eight students, the administrator arrives with a badge and a memo that do not match. You talk to him with `T` (`AdminTalk`): each answer has a follow-up that can expose a lie, and every lie exposed takes 15 seconds off the night (never below 120). Only **DECLINE** starts the hunt. ACCEPT loses at once. The hunt (`AdminHunt` state machine, shown by `AdminHuntDirector`) lasts 210 seconds: he stops in a room (keep the lamp on his head for 1.8 s within 9 s) or behind you (flash his face within 2.8 s). Each success gives back 8 sanity, each catch costs 15, and 3 catches lose. He stops more often as dawn nears. **Doors are not safe:** if you leave him in his room, that door is a catch next time. **Night 6** is the same with 300 seconds, 2 catches, faster run-ups, and in

a room he circles and dodges the lamp. **The hunt can never get stuck:** the dawn clock always runs and every phase has a time limit, and a test proves it with random and idle players.

Lockdown

One finite night. All 13 students arrive 8 to 11 seconds apart, 6 of them anomalies, all frightened. Each face gets 35 seconds, then it is taken. Three mistakes (an anomaly let in, a student refused, a student taken) lose. No patrol, no sanity drain. It never changes your story save. Rules are in `GameLogic/Lockdown.cs`.

Saving

`SaveFormat` writes a text file with the header `SMYIDSAVE`, **version 4** and a checksum. `SaveSystem` writes a temp file then swaps it in (keeping a backup), so a crash mid-save cannot corrupt it. A corrupt or old-version save is ignored instead of crashing the game. The start of every night is also kept in its own file, so RETRY is always the true start of that night. Files live in Unity's persistent data folder (on Windows: `AppData\LocalLow\Twin Studio\Show Me Your ID!`). Volumes and settings go in PlayerPrefs, and Night 6 / Lockdown unlocks do too.

Randomness and fairness

Every new game gets a random seed that is saved with the run (`GameRng`). Continue rebuilds the same visitors from the seed, so the save only needs a few numbers, not the whole roster. Tests can fix the seed to replay a night exactly.

Config

Nothing about the rules is buried in code. `GameRules.cs` holds each rule number once, `GameRules.Tables.cs` holds the nights and students as rows, and the `GameConfig.asset` holds the live tuning copy. If the asset is missing, the game still runs on the defaults (`GameConfigSO.LoadOrDefault`).

Input, art and audio

- **Input:** the Input System package with Active Input Handling set to Both. Gameplay keys and the mouse are read with the classic `Input` class (`Input.GetKey`), and uGUI clicks go through the Input System UI Input Module that `GameCore` adds to the EventSystem. `InputGate` makes sure one view owns a key per frame (Esc and M are shared).
- **Art:** `ProceduralArt.Get(key)` loads `Resources/Art/<key>` first. Only if no file exists does it paint a placeholder in code. Some screens still use placeholders (the full list is in `docs/ASSET-MANIFEST.md`).
- **Audio:** `AudioLibrary` loads `Resources/Audio/<key>`. There are **no audio files yet**, so the game is silent. Important moments are covered by on-screen captions.

---

7. How we know it works (tests and tools)

Run everything with one command: `bash Tools/run_all.sh`. The last run on November 3, 2026 printed:

Step	Result
Rules tests (`Tools/LogicTests`)	**120,438 checks, 0 failed**
Flow check (`Tools/FlowCheck`): presses real buttons headlessly through the desk, patrol, hunt, night 6, lockdown, saves and retries	**1,463 OK, 0 bad**
Compile against the real Unity assemblies (`Tools/Verify`)	PASS
No em dashes anywhere (`Tools/check_no_em_dashes.py`, a team writing rule)	PASS

Be honest about the limit: these checks cannot judge how the game looks, sounds or feels, and the Set Up Project tool only runs inside Unity. Those were checked by playing in the editor.

In the Unity editor menu (Hollowbrook):

- **Set Up Project (one click):** fills the three scenes with the real UI objects, lists them in Build Settings (MenuScene first), sets the art import settings and makes Play start at the main menu. It only adds what is missing and never overwrites your edits.
- **Check Project:** only reports problems.
- **Rebuild Scenes From Scratch:** asks first, then throws a scene's contents away and refills it.

---

8. The Game Design Document matches the build (v2.6)

The document (`Show-Me-Your-ID-GDD-v2.6`) was corrected to the build on November 3, 2026. These are the rules people most often get wrong, so have them ready:

Topic	What the game (and the document) says
Visitors per night	4, 6, 7, 8, then 8 students + the administrator (34 total)
Window anomalies	Fixed: 2, 3, 4, 5, 5
Stamping the administrator	DECLINE closes the window and the Patrol button starts the hunt. ACCEPT loses at once
Patrol	Opens after the last visitor of the night. Night 5 has no patrol cycle: the hunt is the patrol
Failing a night	Ending a shift with a breach fails the night and costs sanity on the next one. A blackout replays it. Breaches and false rejections are totals for the whole game, and 3 lose on night 5
Students	13 (Izzy exists in the code but is switched off)
Tutorial	Built but switched off, night 1 plays like any other night
Clue order	A clue found out of order still counts as found, but the Secret ending is then out of reach for that game

The older version 2.5 of the document had different design numbers (4, 5, 6, 7 visitors, both stamps starting the hunt). If someone has that copy, say: "That was the earlier design. Version 2.6 is the build."

---

9. Unfinished or imperfect (say it before they find it)

- **Audio:** no sound files yet. The game is silent. Captions carry the cues.

- **Some art is still code-drawn:** the map sheets, some paper pieces, the cursor and effect layers (listed in `docs/ASSET-MANIFEST.md`). The jumpscare art folder is empty, so scares zoom the anomaly's own picture.
- **Nothing was balanced by outside players.** The speeds, timers and sanity numbers are our best guesses, all editable in `GameConfig` and `GameRules`.
- **The patrol world is not stored in scene files.** Rooms, doors and clues are built from code each time.
- **No prefabs for screens yet.** The screens are objects in the scene files, built by code the first time.
- **The camera, controller and audio cannot be edited visually** (part 1).
- **Windows PC only,** mouse and keyboard, single player, English text only.

---

10. Likely questions and short answers

The questions you asked about

Why is there no camera in the Hierarchy? (Part 1.) Created by code at startup, survives every scene, so there is only one. Press Play to see it under DontDestroyOnLoad.

Why is the Hierarchy almost empty in Edit mode? Because most of the game is made at runtime: visitors, papers, the guard, the rooms, the anomalies. Only the screens (canvases, buttons, labels) are saved in the scene, and only after Set Up Project.

How does the player move with no physics? Position changes in `PlayerController`. Doors and items are found by distance along the hallway. We did not need collisions, and it keeps the game simple to test.

How do clicks work? An `EventSystem` (created in code) plus the uGUI canvases. Each button is wired in code to a method on a view or on `GameRoot`.

Why three scenes? The desk is only interface, the patrol is a world of sprites with a moving camera, and the menu is its own thing. Splitting them keeps each scene simple and lets the patrol load only when needed. `GameRoot` carries the run across.

Why build the interface in code if it is also in the scene? It started as code only. Artists could not edit it, so we added Set Up Project, which saves the same objects into the scene. The code finds them first and only makes what is missing, and `SceneEdits` keeps your changes on top.

About the code

What is the core idea? Rules and screens are separate. Rules are plain C# that answers questions, screens draw the answers. That is why 120,000 checks run without opening Unity.

What is `GameRoot`? The one object that survives scene changes. It holds the session, saves, audio, settings, the queue and links to every screen. Gameplay code never searches by name.

What is a partial class and why use it? One class written across files. `GameRoot` is big, so each file holds one topic. The compiler treats it as one class.

What is the EventBus for? So parts do not need to know each other. One publishes by id, others subscribe.

Why a `ScriptableObject` for the config? (`GameConfigSO`) So tuning numbers live in one asset a designer can edit in the Inspector, with no code change.

Why plain C# in `GameLogic`? So it runs in tests with no Unity. It also keeps rules from being tangled up with drawing.

How does saving work and what if it breaks? Text file with a header, version and checksum, written to a temp file and swapped in with a backup. A corrupt or old file is ignored, so the game starts fresh instead of crashing.

About the gameplay

Is it random or the same each time? Different each new game (random seed), but the seed is saved so a night can be rebuilt exactly. Every student takes a turn as an anomaly (whoever has been one the fewest times goes next).

How do you decide someone is an anomaly? `RosterFactory` picks them and applies defects. `IdRules` finds them. Your stamp is compared with that truth.

Can a night be unwinnable? No. Every anomaly has a findable tell (tests prove it), the hunt has time limits in every phase, and a test runs random and idle players to prove the hunt always ends.

What does the flashlight do? Maths picks a cone. The same cone clears the darkness and decides what is slowed or repelled. Sweep is wide and only slows. Focus is tight and repels.

What is Night 6? Lockdown? (Part 6.)

What is the pocket watch? Clue 7. It is the game's only token, can be inspected once, and has no other use.

About the project

Hardest bug? Good answer: the Night 5 and 6 softlock. Opening the map switched off the stamp sequence, so the night froze. The fix made the sequence wait for the map. Also the captions bug: every caption showed as an empty black strip because the words were sent in one field of the event while the bar read another.

What would you do next? Final audio, finish the code-drawn art, outside playtesting to tune the numbers, move more of the patrol into scenes, and decide which of the document's and the build's rules win.

Who made what? Project Director and narrative Jessie Mariel De Mesa. Programming Elijah Japeth Ramos and John Perilla. Art Daniel Steven Solitaria and Jessie Mariel De Mesa. Audio Clarence Manalo. Marketing and branding Diosdado Fabelico II and Daniella Garing.

If you do not know: say which layer it lives in and which file, then "I would need to open it to be exact". That is true and it shows you understand the structure.

---

11. Numbers cheat sheet (all checked in the code)

Thing	Value
Engine	Unity 6.3 (6000.3.0f1), Built-in Render Pipeline, uGUI, Input System 1.20.1 (Active Input Handling: Both)
Scripts / lines	130 files, about 22,600 lines
Scenes	3 (MenuScene, DeskScene, PatrolScene)
Camera	Orthographic, size 5, black, at (0, 0, -10)
Canvas reference	1280 x 720, three canvases: FX (order 5), UI (10), Scare (20)
Nights	5 story nights, plus Night 6 and Lockdown
Visitors	4, 6, 7, 8, then 8 students + administrator
Anomalies at the window	2, 3, 4, 5, 5
Students	13
Questions	4 exist, 2 allowed per student
Defect types	13
Anomaly behaviors	4 (Blink, Slide, Sprint, CoverHug)
Clues	10, in order, for the Secret ending
Arrival gap	25 to 30 s on night 1, times 0.8 each night
Sanity stages	Stable 70+, Uneasy 40+, Shaken 15+, Breaking below
Hunt (night 5)	210 s, 3 catches, lamp 1.8 s within 9 s, flash within 2.8 s, 15 s off per lie (min 120 s)
Night 6	300 s, 2 catches, 10 s off per lie (min 180 s)
Lockdown	13 students, 6 anomalies, 35 s each, 3 mistakes, arrivals 8 to 11 s
Save	text file, header SMYIDSAVE, version 4, checksum
Tests	120,438 rules checks, 1,463 flow checks, plus the real Unity compile

---

12. A demo script and a study plan

Demo (3 minutes):

1. Show the Hierarchy in Edit mode (empty of camera). Press Play and show `DontDestroyOnLoad` (part 1).
2. NEW GAME. Check one ID, ask two questions, stamp. Say "the stamp is permanent and compared with the truth".
3. Press PATROL. Show the flashlight, peek into a room, and clear a figure.
4. Open `Resources/GameConfig.asset` and change one number (for example `huntSurviveSeconds`). Say "no code change needed".
5. Terminal: `bash Tools/run_all.sh` and show it all pass.

Study plan (4 short sessions):

1. 30 min: parts 1 to 3 out loud. Draw the layers table from memory.
2. 45 min: open `GameCore.cs`, `GameSession.cs`, `ShiftPlan.cs` and `EndingRules.cs`. Explain each in two sentences.
3. 45 min: follow `StartNewGame` to `BeginShift` in `GameRoot.Shifts.cs` with your finger, then read the `HuntPhase` enum in `AdminHunt.cs` and say the phases without looking.
4. 30 min: run `bash Tools/run_all.sh`, then read parts 8 and 10 as flash cards.